RESEARCH REPORT · DEEP LEARNING

Artificial Neural Network Weight Initialisation

A Mathematical Proof of the Symmetry Problem in Sigmoid Under Zero Weight Initialisation and Comparison with Tanh, and ReLU

Samriddha Raj Satyal

Independent Research

Artificial Neural Networks — Backpropagation Theory

Status: Final

Abstract

This report presents a step-by-step mathematical proof examining the behaviour of three widely-used activation functions — Sigmoid, Tanh, and ReLU — under zero weight initialization in a two-layer artificial neural network. Gradient expressions is derived using the chain rule of derivatives, and different cases are demonstrated: training failure, symmetry occurance, and normal behaviour. Further, theoretical findings are validated with empirical results and identify the exact mechanism by which Sigmoid produces a symmetric (linear) model while Tanh and ReLU become completely dead. This analysis clarifies several vague claims found in existing literature.

Contents

Abstract **1**

1 Introduction **3**

2 Network Architecture and Notation **3**
2.1 Forward Pass Equations 4
2.2 Activation Functions Under Analysis 4

3 Gradient Derivations via the Chain Rule **4**
3.1 Output Layer Gradients 4
3.2 Hidden Layer Gradients 5

4 Analysis Under Zero Initialisation **6**
4.1 Initial State (Step 0) 6
4.2 Step 1: First Gradient Computation 6
4.2.1 Output bias gradient — identical for all three: 6
4.2.2 Output weight gradient — the divergence point: 6
4.2.3 Hidden weight gradient at Step 1: 7
4.3 Step 2: The Cascade 7

5 The Cascade Mechanism **9**
5.1 Sigmoid: Cascade to Symmetry 9
5.2 Tanh / ReLU: Permanent Deadlock 9

6 Complete Case Analysis **10**

7 Empirical Validation **10**
7.1 Observed Weights After 100 Epochs — Sigmoid 11
7.2 Observed Weights After 100 Epochs — Tanh and ReLU 12

8 Why Textbook Accounts Are Imprecise **12**

9 Conclusion **13**

References **14**

1 Introduction

Weight initialization is a critical component in training deep neural networks. A poor initialization strategy can prevent learning entirely, cause symmetry among neurons, or lead to vanishing and exploding gradients. The simplest approach — initializing all weights and biases to zero — is commonly warned against in textbooks and research literature. However, the precise mathematical reasons for why zero initialization fails, and how different activation functions respond differently, are often stated imprecisely or in a vague manner without in-depth mathematical proof.

This report addresses that gap. We consider a simple two-layer network with two hidden neurons and one output neuron and derive all gradient expressions analytically. We then trace exactly what happens at each training step under zero initialization for Sigmoid, Tanh, and ReLU activations. The findings both confirm and refine claims commonly found in the literature.

2 Network Architecture and Notation

The following architecture is used throughout:

- **Input:** x_1, x_2 (two features)
- **Hidden layer:** 2 neurons — weights $w_{11}^{(1)}$, $w_{21}^{(1)}$, $w_{12}^{(1)}$, $w_{22}^{(1)}$; biases b_{11}, b_{12}
- **Output layer:** 1 neuron — weights $w_{11}^{(2)}$, $w_{21}^{(2)}$; bias b_2
- **Activation function:** $f(\cdot)$, identical for both hidden neurons
- **Loss function:** Mean Squared Error, $L = (\hat{y} - y)^2$

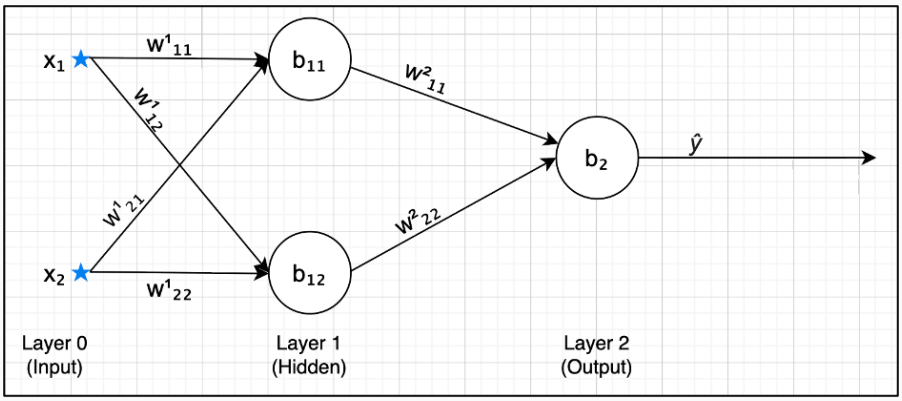

Figure 1: Two-layer network architecture used throughout this analysis.

2.1 Forward Pass Equations

Pre-activation values at the hidden layer:

$$z_{11} = x_1\, w_{11}^{(1)} + x_2\, w_{21}^{(1)} + b_{11} \tag{1}$$

$$z_{12} = x_1\, w_{12}^{(1)} + x_2\, w_{22}^{(1)} + b_{12} \tag{2}$$

Post-activation outputs:

$$O_{11} = f(z_{11}) \tag{3}$$

$$O_{12} = f(z_{12}) \tag{4}$$

Final network output:

$$\hat{y} = O_{11} \cdot w_{11}^{(2)} + O_{12} \cdot w_{21}^{(2)} + b_2 \tag{5}$$

2.2 Activation Functions Under Analysis

Activation	Formula	$f(0)$	$f'(0)$
Sigmoid	$\dfrac{1}{1+e^{-z}}$	0.5	0.25
Tanh	$\dfrac{e^z - e^{-z}}{e^z + e^{-z}}$	0	1
ReLU	$\max(0,\, z)$	0	0^*

Table 1: Activation functions and their critical values at $z = 0$. ReLU is not differentiable at $z = 0$; conventionally $f'(0) = 0$ is used.

3 Gradient Derivations via the Chain Rule

The following gradients form the backbone of backpropagation. We derive all of them explicitly from first principles.

3.1 Output Layer Gradients

The dependency chain for $w_{11}^{(2)}$ is $L \to \hat{y} \to w_{11}^{(2)}$. Since $\hat{y} = O_{11} \cdot w_{11}^{(2)} + O_{12} \cdot w_{21}^{(2)} + b_2$:

$$\frac{\partial L}{\partial w_{11}^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_{11}^{(2)}} = 2(\hat{y} - y) \cdot O_{11} \tag{6}$$

$$\frac{\partial L}{\partial w_{21}^{(2)}} = 2(\hat{y} - y) \cdot O_{12} \tag{7}$$

$$\frac{\partial L}{\partial b_2} = 2(\hat{y} - y) \tag{8}$$

Observation. $w_{11}^{(2)}$ vanishes upon differentiation — the gradient $\frac{\partial L}{\partial w_{11}^{(2)}}$ depends on O_{11}, *not* on $w_{11}^{(2)}$ itself. Even if $w_{11}^{(2)} = 0$, this gradient can be non-zero provided $O_{11} \neq 0$.

3.2 Hidden Layer Gradients

The extended chain for $w_{11}^{(1)}$ is $L \to \hat{y} \to O_{11} \to z_{11} \to w_{11}^{(1)}$:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial z_{11}} \cdot \frac{\partial z_{11}}{\partial w_{11}^{(1)}} = 2(\hat{y} - y) \cdot w_{11}^{(2)} \cdot f'(z_{11}) \cdot x_1 \tag{9}$$

Expanding to the full set:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = 2(\hat{y} - y) \cdot w_{11}^{(2)} \cdot f'(z_{11}) \cdot x_1 \tag{10}$$

$$\frac{\partial L}{\partial w_{21}^{(1)}} = 2(\hat{y} - y) \cdot w_{11}^{(2)} \cdot f'(z_{11}) \cdot x_2 \tag{11}$$

$$\frac{\partial L}{\partial w_{12}^{(1)}} = 2(\hat{y} - y) \cdot w_{21}^{(2)} \cdot f'(z_{12}) \cdot x_1 \tag{12}$$

$$\frac{\partial L}{\partial w_{22}^{(1)}} = 2(\hat{y} - y) \cdot w_{21}^{(2)} \cdot f'(z_{12}) \cdot x_2 \tag{13}$$

Key multiplier: The output layer weights $w_{11}^{(2)}$ and $w_{21}^{(2)}$ appear as multipliers in every hidden layer gradient. If either is zero, the entire corresponding hidden gradient collapses to zero regardless of the activation function or input values.

4 Analysis Under Zero Initialisation

All weights and biases are set to zero; every step of training is traced analytically.

4.1 Initial State (Step 0)

$$z_{11} = z_{12} = x_1 \cdot 0 + x_2 \cdot 0 + 0 = 0 \tag{14}$$

Activation	$O_{11} = f(0)$	$f'(0)$	$\hat{y}$	L
Sigmoid	0.5	0.25	$0.5(0) + 0.5(0) + 0 = 0$	$(0-y)^2$
Tanh	0	1	$0(0) + 0(0) + 0 = 0$	$(0-y)^2$
ReLU	0	0	$0(0) + 0(0) + 0 = 0$	$(0-y)^2$

Table 2: All three activations produce $\hat{y} = 0$ at initialisation.

4.2 Step 1: First Gradient Computation

4.2.1 Output bias gradient — identical for all three:

$$\frac{\partial L}{\partial b_2} = 2(\hat{y} - y) = -2y \neq 0 \quad \text{(assuming } y \neq 0\text{)} \tag{15}$$

The output bias updates for *all three* activations.

4.2.2 Output weight gradient — the divergence point:

From equation (6):

$$\frac{\partial L}{\partial w_{11}^{(2)}} = 2(0-y) \cdot O_{11} = -2y \cdot O_{11} \tag{16}$$

Activation	O_{11}	$\partial L / \partial w_{11}^{(2)}$	$w^{(2)}$ updates?
Sigmoid	0.5	$-2y \times 0.5 \neq 0$	**Yes — at Step 1**
Tanh	0	$-2y \times 0 = 0$	**No — permanently stuck**
ReLU	0	$-2y \times 0 = 0$	**No — permanently stuck**

Table 3: Output weight gradient at Step 1 — the single structural divergence among the three activations.

☆ Key Insight

Sigmoid's hidden output of **0.5** — rather than 0 — is the sole reason it escapes total deadlock. The output weights $w^{(2)}$ receive a non-zero gradient and update at Step 1. Tanh and ReLU output 0, which multiplies the entire gradient to zero, permanently blocking $w^{(2)}$ from ever changing.

4.2.3 Hidden weight gradient at Step 1:

Because $w_{11}^{(2)} = 0$ at this point for all three:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = 2(0 - y) \cdot \underbrace{w_{11}^{(2)}}_{=0} \cdot f'(0) \cdot x_1 = 0 \tag{17}$$

Hidden weights remain zero for all three after Step 1. The difference is that Sigmoid's output weights are now non-zero, which will unlock the hidden layer in Step 2.

4.3 Step 2: The Cascade

Activation	$w_{11}^{(2)}$ after Step 1	Hidden gradient unblocked?
Sigmoid	$\neq 0$	**Yes**
Tanh	$= 0$	**No**
ReLU	$= 0$	**No**

Table 4: Output weight state after Step 1 and its effect on the hidden layer.

For **Sigmoid** at Step 2, $w_{11}^{(2)} \neq 0$, so:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = 2(\hat{y} - y) \cdot w_{11}^{(2)} \cdot 0.25 \cdot x_1 \neq 0 \tag{18}$$

Hidden weights finally update. However, because $O_{11} = O_{12} = 0.5$ caused $w_{11}^{(2)}$ and $w_{21}^{(2)}$ to update by *the same amount*, we have $w_{11}^{(2)} = w_{21}^{(2)}$. Consequently:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = 2(\hat{y} - y) \cdot w_{11}^{(2)} \cdot 0.25 \cdot x_1 \tag{19}$$

$$\frac{\partial L}{\partial w_{12}^{(1)}} = 2(\hat{y} - y) \cdot w_{21}^{(2)} \cdot 0.25 \cdot x_1 \tag{20}$$

Since $w_{11}^{(2)} = w_{21}^{(2)}$:

$$\boxed{\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial w_{12}^{(1)}}} \tag{21}$$

Both hidden neurons receive identical gradients, update identically, and remain permanently symmetric — the network behaves as a **single-neuron linear model**.

5 The Cascade Mechanism

5.1 Sigmoid: Cascade to Symmetry

$$O_{11} = O_{12} = 0.5 \text{ — neither is zero}$$

$$\frac{\partial L}{\partial w_{11}^{(2)}} = -2y \times 0.5 \neq 0 \quad \text{(equal for both neurons)}$$

$$w_{11}^{(2)} = w_{21}^{(2)} \neq 0 \text{ — updated by equal amounts at Step 1}$$

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial w_{12}^{(1)}} \text{ — identical gradient to both hidden neurons}$$

Symmetry is permanent ⇒ Network reduces to a linear model

5.2 Tanh / ReLU: Permanent Deadlock

$$O_{11} = O_{12} = 0 \text{ — both outputs are zero}$$

$$\frac{\partial L}{\partial w_{11}^{(2)}} = -2y \times 0 = 0 \text{ — gradient killed at source}$$

$$w_{11}^{(2)} = w_{21}^{(2)} = 0 \text{ — output weights never receive a non-zero gradient}$$

$$\frac{\partial L}{\partial w_{11}^{(1)}} = 2(\hat{y}-y)\times 0 \times f'(0) \times x_1 = 0 \text{ — hidden gradient also zero}$$

Complete deadlock ⇒ No parameter ever updates

6 Complete Case Analysis

Condition	Sigmoid	Tanh	ReLU
All weights $= 0$**, all biases** $= 0$	✗ **No**	✗ **No**	✗ **No**
Hidden $w^{(1)} = 0$**, output** $w^{(2)} = 0$ **(Step 1)**	**Updates**	**Stuck**	**Stuck**
After Step 1	$\sim$ **Partial** (symmetry/linear)	✗ **No**	✗ **No**
Random init, normal z **values**	✓ **Yes**	✓ **Yes**	✓ **Yes**
Random init, very large $\lvert z \rvert$	✗ **No** (vanish)	✗ **No** (vanish)	✓ **Yes**
Random init, very negative z	✗ **No** (vanish)	✗ **No** (vanish)	✗ **No** (dead)

Table 5: Complete case analysis across all initialisation conditions and activation functions.

7 Empirical Validation

Theoretical predictions were validated with the following Keras experiment:

```python
model = Sequential()
model.add(Dense(2, activation='sigmoid', input_dim=2))
model.add(Dense(1, activation='sigmoid'))

# Force all weights and biases to zero
initial_weights = model.get_weights()
initial_weights[0] = np.zeros(initial_weights[0].shape)  # hidden weights
initial_weights[1] = np.zeros(initial_weights[1].shape)  # hidden biases
initial_weights[2] = np.zeros(initial_weights[2].shape)  # output weights
initial_weights[3] = np.zeros(initial_weights[3].shape)  # output bias
model.set_weights(initial_weights)

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
history = model.fit(X, y, epochs=100, validation_split=0.2)
```

7.1 Observed Weights After 100 Epochs — Sigmoid

$$\text{Hidden weights:} \quad \begin{bmatrix} -0.3665 & -0.3665 \\ 0.3798 & 0.3798 \end{bmatrix}$$

$$\text{Hidden biases:} \quad \begin{bmatrix} -0.0280 & -0.0280 \end{bmatrix}$$

$$\text{Output weights:} \quad \begin{bmatrix} -0.0549 \\ -0.0549 \end{bmatrix}$$

$$\text{Output bias:} \quad \begin{bmatrix} 0.0048 \end{bmatrix}$$

☆ Key Insight

Perfect symmetry confirmed: $w_{11}^{(1)} = w_{12}^{(1)}$, $w_{21}^{(1)} = w_{22}^{(1)}$, $w_{11}^{(2)} = w_{21}^{(2)}$. Both neurons learned exactly the same weights at every epoch, confirming the symmetry proof analytically and empirically. The network is equivalent to a single-neuron model — precisely as predicted.

7.2 Observed Weights After 100 Epochs — Tanh and ReLU

All weights and biases remain exactly 0.000 after 100 epochs. No learning occurred whatsoever, confirming that $w^{(2)} = 0$ permanently blocks all hidden gradients and renders the network inert.

8 Why Textbook Accounts Are Imprecise

The literature typically states:

- *"Tanh and ReLU have dead gradients under zero initialisation."*
- *"Sigmoid becomes a linear model under zero initialisation."*

These statements describe structurally *different* failure modes as if they are the same phenomenon. The analysis here reveals the precise mechanistic distinction:

Textbook Claim	Precise Mechanism	Assessment
Tanh/ReLU: dead gradients	$f(0) = 0$ kills $\frac{\partial L}{\partial w^{(2)}}$; cascade propagates to all $\frac{\partial L}{\partial w^{(1)}}$	Correct — under-explained
Sigmoid: linear model	$f(0) = 0.5$ permits $w^{(2)}$ to update, but equally for both neurons, inducing symmetry	Correct — imprecisely stated
Sigmoid "differs" from Tanh/ReLU	True only because $f(0) = 0.5$ vs. $f(0) = 0$; the mechanism *is* the difference	Partially correct
Zero init is catastrophic for all	True, but for structurally distinct reasons depending on the activation	Correct conclusion

Table 6: Evaluation of common textbook claims against the rigorous derivation.

9 Conclusion

1. Under complete zero initialisation, *all three* activations fail identically at Step 1: every gradient is zero and no parameter update occurs.

2. The critical divergence arises at the **output weight gradient**. Sigmoid's $f(0) = 0.5$ produces a non-zero gradient at Step 1; Tanh and ReLU's $f(0) = 0$ permanently kills it.

3. This single structural difference triggers a cascade: Sigmoid's output weights update $\Rightarrow$ hidden weights become updatable $\Rightarrow$ but *symmetrically*, collapsing the network to a linear model. Tanh/ReLU's output weights never update $\Rightarrow$ hidden weights are permanently frozen.

4. Symmetry can be broken for Sigmoid by initialising output weights unequally (i.e., random initialisation), demonstrating that the failure is an artefact of the *initialisation strategy*, not the activation function per se.

5. The universal prescription is **random weight initialisation** (e.g., Xavier/Glorot for Sigmoid/Tanh, He initialisation for ReLU), which breaks symmetry and guarantees non-zero gradients from the very first step.

References

[1] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of AISTATS*.

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *ICCV*.

[3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[4] LeCun, Y., Bottou, L., Orr, G., & Müller, K. (1998). Efficient BackProp. In *Neural Networks: Tricks of the Trade*. Springer.

[5] Rumelhart, D., Hinton, G., & Williams, R. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

[6] CampusX. (n.d.). *Weight Initialisation Techniques — What not to do? — Deep Learning*. YouTube. https://youtu.be/2MSY0HwH5Ss